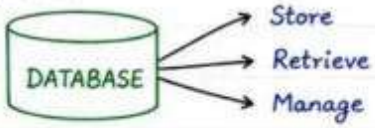
DATABASE FUNDAMENTALS

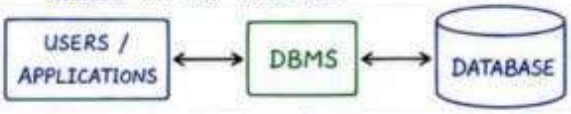
1. What is a Database?

- A database is an organized collection of data that can be easily accessed, managed, and updated.



2. What is DBMS?

- DBMS (Database Management System) is software that allows users to define, create, maintain, and control access to the database.



3. What is RDBMS?

- RDBMS (Relational Database Management System) stores data in tables (relations) with rows and columns and maintains relationships between tables.

ID	Name	Age	City
1	Alice	20	Hyd
2	Bob	21	Blr
3	Charlie	22	Vizag

Data stored in tables (Relations)

4. DBMS vs RDBMS

DBMS	RDBMS
• General term for any Database Management System.	• Type of DBMS that uses relational model (tables).
• Can be Hierarchical, Network, Object-oriented etc.	• Uses tables with rows and columns.
• May not support relationships.	• Supports relationships using keys.
• Examples: File System, XML DBMS	• Examples: MySQL, PostgreSQL, Oracle, SQL Server.

5. What is SQL?

- SQL (Structured Query Language) is a standard language used to communicate with databases.

Used for: Creating, Reading, Updating, Deleting and Managing data.

6. What is MySQL?

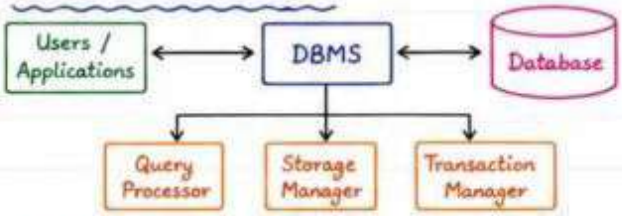
- MySQL is an open-source RDBMS that uses SQL for managing data.
- It is fast, reliable and widely used in web applications.



7. SQL vs MySQL

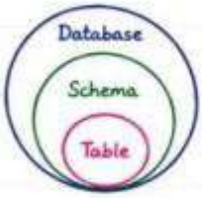
SQL	MySQL
• It is a language. • Used to access and manipulate data. • Standard (ANSI). • Examples: SELECT, INSERT, UPDATE, DELETE	• It is a database management system. • Uses SQL as its query language. • Open-source RDBMS. • Example: MySQL Server

8. Database Architecture



9. Database, Schema, Table

- Database: A collection of related data.
- Schema: Logical structure of the database. It contains tables, relationships, views, etc.
- Table: A collection of related data in rows and columns.



10. Rows and Columns

- Table data is stored in rows and columns.

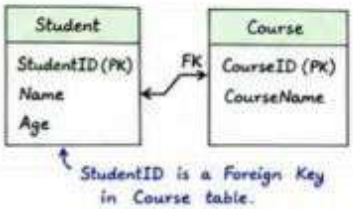
ID	Name	Age	City
1	Alice	20	Hyd
2	Bob	21	Blr
3	Charlie	22	Vizag

Column (Vertical)

Row (Horizontal)

11. Relational Database Concepts

- Table (Relation): Data is stored in tables.
- Primary Key: Uniquely identifies each row in a table.
- Foreign Key: Links one table to another.
- Relationship: Connection between two tables.
- Integrity: Accuracy and consistency of data.
- Normalization: Organizing data to reduce redundancy.



Remember: Good data → Good information → Good decisions 😊

Keep Practicing!

★ DATA TYPES ★

1. Numeric Data Types

- Used to store numbers (with or without decimals).

① Integer Types

- Store whole numbers (... -2, -1, 0, 1, 2...).

123

Data Type	Size	Range (Signed)	Range (Unsigned)
TINYINT	1 byte	-128 to 127	0 to 255
SMALLINT	2 bytes	-32,768 to 32,767	0 to 65,535
MEDIUMINT	3 bytes	-8,388,608 to 8,388,607	0 to 16,777,215
INT / INTEGER	4 bytes	-2,147,483,648 to 2,147,483,647	0 to 4,294,967,295
BIGINT	8 bytes	Very Large Range	Very Large Range

② Fixed-Point Types

- Store exact decimal numbers.
  Used for precise values like money.



Data Type	Description	Example
DECIMAL(M,D)	M = total digits, D = digits after decimal	DECIMAL(10,2) 12345.67

③ Floating-Point Types

- Store approximate decimal numbers.
  Used for scientific calculations.

3.14

Data Type	Size	Description	Example
FLOAT(p)	4 bytes	Single precision	3.14159
DOUBLE(p)	8 bytes	Double precision	3.1415926535
LONGBLOB(p)	8 bytes	Extended precision	3.141592653589

2. String Data Types

- Used to store text or combination of characters.

A B C

① Character Types

- Store fixed length character strings.

Data Type	Description	Example
CHAR(n)	Fixed length string (n characters)	CHAR(5) → 'HELLO'
BINARY(n)	Fixed length binary string	BINARY(4)

② Text Types

- Store variable length text.



Data Type	Description
VARCHAR(n)	Variable length string (max n)
TINYTEXT	Up to 255 characters
TEXT	Up to 65,535 characters
MEDIUMTEXT	Up to 16,777,215 characters
LONGTEXT	Up to 4,294,967,295 characters

Tip: Use VARCHAR for short text (like names, emails)
Use TEXT types for long content (like articles, descriptions)

3. Date & Time Types

- Used to store date and time values.



Data Type	Description	
DATE	Stores date (YYYY-MM-DD)	2024-05-20
TIME	Stores time (HH:MM:SS)	14:30:00
DATETIME	Stores date and time	2024-05-20 14:30:00
TIMESTAMP	Date and time with time zone	2024-05-20 14:30:00
YEAR	Stores year in 4 digits	2024

4. Boolean Type

- Stores true or false values.

☑ TRUE ☐ FALSE

BOOLEAN / BOOL / TINYINT(1) 0 = FALSE, 1 = TRUE

5. Binary Data Types

- Used to store binary data like images, files.



Data Type	Description
BINARY(n)	Fixed length binary string
VARBINARY(n)	Variable length binary string
TINYBLOB	Up to 255 bytes
BLOB	Up to 65,535 bytes
MEDIUMBLOB	Up to 16,777,215 bytes
LONGBLOB	Up to 4,294,967,295 bytes

6. JSON Type

- Stores JSON (JavaScript Object Notation) data.

{ : , }

```
Example:
{"name": "Alice",
"age": 22, "city": "Hyd"}
```

← Flexible format to store structured data.

7. ENUM

- Column has a predefined set of values.

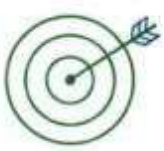
ENUM('Red', 'Green', 'Blue') → Example values: Red, Green or Blue only

8. SET

- Column can have zero or more values from a list.

SET('A', 'B', 'C', 'D') → Example values: 'A,B', 'C,D', 'A,C,D' or NULL

9. Choosing the Correct Data Type



- Use the smallest possible data type.
- Use UNSIGNED for non-negative values.
- Use DECIMAL for money, not FLOAT/DOUBLE.
- Use VARCHAR instead of TEXT if length is known.
- Use appropriate Date / Time type for date and time values.
- Proper choice improves performance and saves storage.

Remember: Right Data Type → Less Storage → Better Performance → Efficient Database



Keep Practicing! ★

SQL SYNTAX & COMMANDS

1. SQL Syntax

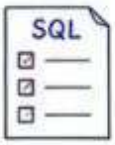
- SQL (Structured Query Language) follows a standard syntax to communicate with databases.

Basic SQL Syntax Structure

```
CLAUSE keyword (in UPPERCASE) + object / expression (table name, column, value) ;
```

Example :

```
SELECT column_name
FROM table_name
WHERE condition;
```



2. SQL Statements

- A statement is a complete instruction that SQL Server can execute.

Examples :

```
CREATE TABLE student (...);
INSERT INTO student VALUES (1, 'Alice');
```

3. Keywords

- Keywords are reserved words in SQL.
- They are used to perform operations.

Examples :

```
SELECT, FROM, WHERE, INSERT, UPDATE,
DELETE, CREATE, DROP, ALTER, AND, OR, NOT,
IN, LIKE, BETWEEN, ORDER BY, GROUP BY
```



4. Identifiers

- Identifiers are names given to database objects like tables, columns, views, etc.

Rules :

- Can contain letters, digits and underscore (_).
- Must start with a letter or underscore.
- Cannot be a keyword.
- Case-insensitive (in MySQL), but lowercase recommended.

Examples :
student, student_info, age1



5. Comments

- Comments are used to explain the code.

Single line comment :

```
-- This is a comment
```

Multi-line comment :

```
/* This is a
   multi-line comment
*/
```



6. SQL Command Categories

- SQL commands are grouped into 5 major categories.



Category	Full Form	Purpose	Example Commands
DDL	Data Definition Language	Define or modify database objects	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	Insert, update, delete data	INSERT, UPDATE, DELETE
DQL	Data Query Language	Retrieve data from database	SELECT
DCL	Data Control Language	Control access and permissions	GRANT, REVOKE
TCL	Transaction Control Language	Manage transactions	COMMIT, ROLLBACK, SAVEPOINT

7. DDL (Data Definition Language)

- Used to define and manage database structures.



Commands :

- CREATE – Create database objects
- ALTER – Modify existing objects
- DROP – Delete objects
- TRUNCATE – Remove all records from a table

Examples :

```
CREATE TABLE student (id INT, name VARCHAR(50));
ALTER TABLE student ADD column age INT;
DROP TABLE student;
TRUNCATE TABLE student;
```

8. DML (Data Manipulation Language)

- Used to manipulate data in tables.

Commands :

- INSERT – Add new data
- UPDATE – Modify existing data
- DELETE – Remove data

Table : student

id	name	age
1	Alice	20
2	Bob	21
3	Charlie	22

Examples :

```
INSERT INTO student VALUES (1, 'Alice', 20);
UPDATE student SET age = 21 WHERE id = 1;
DELETE FROM student WHERE id = 3;
```

9. DQL (Data Query Language)

- Used to retrieve data from database.



Command :

- SELECT – Fetch data based on conditions

Example :

```
SELECT id, name FROM student WHERE age > 20;
```

10. DCL (Data Control Language)

- Used to control access and permissions.



Commands :

- GRANT – Give privileges
- REVOKE – Remove privileges

Example :

```
GRANT SELECT ON student TO user1;
REVOKE INSERT ON student FROM user1;
```

11. TCL (Transaction Control Language)

- Used to manage transactions.

Commands :

- COMMIT – Save changes
- ROLLBACK – Undo changes
- SAVEPOINT – Set a save point

Example :

```
COMMIT;
ROLLBACK;
SAVEPOINT sp1;
```



Quick Tip

- SQL keywords in UPPERCASE improve readability.
- Always end a statement with ;

Remember: Practice SQL daily → Master the queries → Build real-world projects!



Keep Practicing!

DATABASE & TABLE MANAGEMENT

1. CREATE DATABASE

- Used to create a new database.
- Once created, we can store tables inside it.

Syntax :

```
CREATE DATABASE db_name;
```

Example :

```
CREATE DATABASE college_db;
```



2. DROP DATABASE

- Used to permanently delete an existing database.
- All tables and data inside it will be lost.

Syntax :

```
DROP DATABASE db_name;
```

Example :

```
DROP DATABASE college_db;
```



3. USE DATABASE

- Used to select a database to work with.

Syntax :

```
USE db_name;
```

Example :

```
USE college_db;
```



4. SHOW DATABASES

- Used to display the list of all databases present in the server.

Syntax :

```
SHOW DATABASES;
```

Example :

```
SHOW DATABASES;
```

Databases
information_schema
mysql
performance_schema
college_db
test
...

5. CREATE TABLE

- Used to create a new table inside the selected database.

Syntax :

```
CREATE TABLE table_name (
  column1 datatype constraints,
  column2 datatype constraints,
  ...
);
```



Example :

```
CREATE TABLE student (
  id INT PRIMARY KEY,
  name VARCHAR(50),
  age INT,
  city VARCHAR(30)
);
```

6. SHOW TABLES

- Used to display the list of tables in the current database.

Syntax :

```
SHOW TABLES;
```

Example :

```
SHOW TABLES;
```

Tables
student
teacher
course
enrollment

7. DESCRIBE

- Used to display the structure (columns and data types) of a table.



Syntax :

```
DESCRIBE table_name;
```

Example :

```
DESCRIBE student;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	
name	varchar(50)	YES		NULL	
age	int	YES		NULL	
city	varchar(30)	YES		NULL	

8. ALTER TABLE

- Used to modify the structure of an existing table.
- Common operations :
  - Add a column
  - Modify a column
  - Drop a column



Syntax & Examples :

```
-- Add Column
ALTER TABLE student ADD email VARCHAR(50);

-- Modify Column
ALTER TABLE student MODIFY age INT NOT NULL;

-- Drop Column
ALTER TABLE student DROP COLUMN city;
```

9. RENAME TABLE

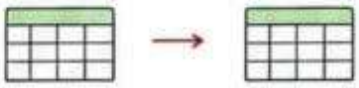
- Used to rename an existing table.

Syntax :

```
RENAME TABLE old_name
  TO new_name;
```

Example :

```
RENAME TABLE student
  TO students;
```



10. DROP TABLE

- Used to delete an existing table and its data permanently.

Syntax :

```
DROP TABLE table_name;
```

Example :

```
DROP TABLE student;
```



11. TRUNCATE TABLE

- Used to remove all records from a table, but the table structure remains.
- Faster than DELETE without WHERE.

Syntax :

```
TRUNCATE TABLE table_name;
```

Example :

```
TRUNCATE TABLE student;
```



Important Notes

- ☆ DROP DATABASE / DROP TABLE → Deletes completely (cannot be undone).
- ☆ TRUNCATE TABLE → Removes all rows but keeps the table.
- ☆ USE database_name; must be used before working with tables.
- ☆ Always be careful while using DROP and TRUNCATE commands! ⚠

Remember: Manage your databases well, so your data stays safe and organized! ☺

Keep Practicing! ☆

DATA MANIPULATION

1. INSERT

- Used to insert new data into a table.
- Adds one row at a time.

Syntax :

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

Example :

```
INSERT INTO student (id, name, age, city)
VALUES (1, 'Alice', 20, 'Hyderabad');
```

student

id	name	age	city



New Row

2. INSERT Multiple Rows

- Used to insert multiple rows in one statement.

Syntax :

```
INSERT INTO table_name (col1, col2, ...)
VALUES (val1, val2, ...),
       (val1, val2, ...),
       (val1, val2, ...);
```

Example :

```
INSERT INTO student (id, name, age, city) VALUES
(2, 'Bob', 21, 'Bangalore'),
(3, 'Charlie', 22, 'Vizag');
```

student

id	name	age	city
1	Alice	20	Hyd
2	Bob	21	Blr
3	Charlie	22	Vizag



Multiple Rows

3. INSERT with Selected Columns

- Insert data into specific columns only.

Syntax :

```
INSERT INTO table_name (col1, col3, ...)
VALUES (val1, val3, ...);
```

Example :

```
INSERT INTO student (name, city)
VALUES ('David', 'Delhi');
```

student

id	name	age	city
4	David	NULL	Delhi

If column is not specified, it will take DEFAULT or NULL.



4. UPDATE

- Used to modify existing data in a table.
- Updates one or more rows.

Syntax :

```
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE condition;
```

Example :

```
UPDATE student
SET age = 21
WHERE name = 'Alice';
```

Before

id	name	age	city
1	Alice	20	Hyd



After

id	name	age	city
1	Alice	21	Hyd

5. UPDATE Multiple Rows

- Update more than one row using a condition.

Syntax :

```
UPDATE table_name
SET column = value
WHERE condition;
```

Example :

```
UPDATE student
SET city = 'Hyderabad'
WHERE age >= 20;
```

Before

id	name	age	city
1	Alice	20	Hyd
2	Bob	19	Blr
3	Charlie	22	Vizag



After

id	name	age	city
1	Alice	20	Hyderabad
2	Bob	19	Blr
3	Charlie	22	Hyderabad

6. DELETE

- Used to delete rows from a table.
- Removes only selected rows.

Syntax :

```
DELETE FROM table_name
WHERE condition;
```

Example :

```
DELETE FROM student
WHERE id = 2;
```

Before

id	name	age	city
1	Alice	20	Hyd
2	Bob	21	Blr
3	Charlie	22	Vizag



After

id	name	age	city
1	Alice	20	Hyd
3	Charlie	22	Vizag

7. DELETE with Conditions

- Always use WHERE clause to delete specific rows.

Syntax :

```
DELETE FROM table_name
WHERE condition;
```

Example :

```
DELECT FROM student
WHERE age < 20;
```



Before

id	name	age	city
1	Alice	18	Hyd
2	Bob	21	Blr
3	Charlie	22	Vizag

After

id	name	age	city
1	Bob	21	Blr
2	Charlie	22	Vizag

8. DELETE vs TRUNCATE vs DROP

Feature	DELETE	TRUNCATE	DROP
Purpose	Deletes selected rows	Deletes all rows from table	Deletes table completely
WHERE clause	Yes	No	No
Rollback	Yes	No	No
Auto Increment reset	No	Yes	Yes (table gone)
Speed	Slow	Fast	Fastest
Structure	Table structure remains	Table structure remains	Table structure removed



DELETE



TRUNCATE



DROP

9. SELECT

- Used to retrieve data from a table.

Syntax :

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Example :

```
SELECT id, name, age FROM student
WHERE age > 20;
```



student

id	name	age	city
1	Alice	20	Hyd
2	Bob	21	Blr
3	Charlie	22	Vizag



Result

id	name	age
2	Bob	21
3	Charlie	22

10. SELECT Specific Columns

- Retrieve only required columns.

Syntax :

```
SELECT column1, column2
FROM table_name;
```

Example :

```
SELECT name, city FROM student;
```

Result

name	city
Alice	Hyd
Bob	Blr
Charlie	Vizag

Tip

Always select only the columns you need. It improves performance!



11. SELECT All Columns

- Retrieve all columns from a table.



Syntax :

```
SELECT * FROM table_name;
```

Example :

```
SELECT * FROM student;
```

Result

id	name	age	city
1	Alice	20	Hyd
2	Bob	21	Blr
3	Charlie	22	Vizag

Remember : Practice Daily → Write Queries → Check Results → Improve Skills





Keep Practicing!



FILTERING & SORTING

Goal: Get the right data in the right order.

WHERE

- Used to filter records that satisfy a condition.

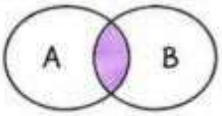
```
SELECT column_names
FROM table_name
WHERE condition;
```

COMPARISON OPERATORS

Operator	Meaning	Example (age = 20)
=	Equal to	age = 20
!= or <>	Not equal to	age != 20
>	Greater than	age > 20
<	Less than	age < 20
>=	Greater than or equal to	age >= 20
<=	Less than or equal to	age <= 20

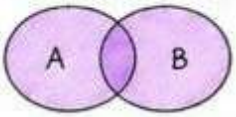
LOGICAL OPERATORS

- AND – Returns true if both conditions are true.



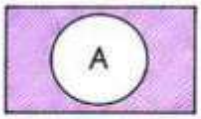
A AND B (Both A and B)

- OR – Returns true if at least one condition is true.



A OR B (A or B or both)

- NOT – Reverses the result.



NOT A (Everything except A)

- IN – Matches any value in a list.

```
WHERE id IN (1, 2, 3)
```
→ id is 1 or 2 or 3

- NOT IN – Excludes values in a list.

```
WHERE id NOT IN (1, 2, 3)
```
→ id is not 1, 2 or 3

- BETWEEN – Between a range (inclusive).

```
WHERE age BETWEEN 18 AND 25
```
→ 18 to 25 included

- NOT BETWEEN – Outside a range.

```
WHERE age NOT BETWEEN 18 AND 25
```
→ age < 18 or age > 25

- LIKE – Searches for a pattern.

```
WHERE name LIKE 'A%'
```
→ Starts with A

- NOT LIKE – Not matching the pattern.

```
WHERE name NOT LIKE '%y'
```
→ Does not end with y

WILDCARDS

Wildcard	Meaning
%	Represents any number of characters (0, 1 or more)
_	Represents a single character

- IS NULL – Checks for missing value.

```
WHERE address IS NULL
```
→ Address is missing

- IS NOT NULL – Checks for available value.

```
WHERE address IS NOT NULL
```
→ Address is available

ORDER BY – Sorts the result set.

- ASC – Ascending order (default)

```
SELECT * FROM students
ORDER BY name ASC;
```

name
Aman
Neha
Riya
Zoya

↑ A to Z

- DESC – Descending order

```
SELECT * FROM students
ORDER BY name DESC;
```

name
Zoya
Riya
Neha
Aman

↓ Z to A

LIMIT – Limits the number of rows returned.

```
SELECT * FROM students
LIMIT 5;
```

Result
1
2
3
4
5

Only first 5 rows

OFFSET – Skips a number of rows before starting.

```
SELECT * FROM students
LIMIT 5 OFFSET 10;
```

Result
11
12
13
14
15

First 10 rows skipped

Next 5 rows

TIPS

- ☑ Use WHERE to filter data.
- ☑ Use ORDER BY to sort data.
- ☑ Use LIMIT and OFFSET for pagination.

Practice more, Write better SQL!

SQL FUNCTIONS

Functions help us perform operations on data and return a single value.

1. Function Fundamentals

- A function is a pre-defined query that performs a calculation and returns a value.
- Functions make queries shorter, easier to read and powerful.
- Functions can be used in SELECT, WHERE, ORDER BY, HAVING etc.

```
SELECT
function_name(column)
FROM table_name;
```



2. Aggregate Functions

- Perform calculations on a set of values and return a single result.

Function	Description	Example
COUNT()	Returns the number of rows.	SELECT COUNT(*) FROM Students;
SUM()	Returns the total sum of values.	SELECT SUM(marks) FROM Students;
AVG()	Returns the average value.	SELECT AVG(marks) FROM Students;
MIN()	Returns the smallest value.	SELECT MIN(marks) FROM Students;
MAX()	Returns the largest value.	SELECT MAX(marks) FROM Students;

3. String Functions

Function	Description	Example
CONCAT()	Joins two or more strings.	SELECT CONCAT('Hello', ' ', 'World'); -- Hello World
LENGTH()	Returns length of a string.	SELECT LENGTH('SQL'); -- 3
LOWER() abc ↓	Converts string to lowercase.	SELECT LOWER('SQL'); -- sql
UPPER() ABC ↑	Converts string to uppercase.	SELECT UPPER('sql'); -- SQL
TRIM() → sql ←	Removes leading and trailing spaces.	SELECT TRIM(' sql '); -- 'sql'
SUBSTRING()	Extracts part of a string.	SELECT SUBSTRING('SQL Tutorial', 1, 3); -- 'SQL'
REPLACE() a → b	Replaces a substring with another.	SELECT REPLACE('abcabc', 'a', 'x'); -- 'xbcxbc'

4. Numeric Functions

Function	Description	Example
ROUND()	Rounds a number to specified decimal places.	SELECT ROUND(12.345, 2); -- 12.35
CEIL()	Returns the smallest integer greater than or equal to the number.	SELECT CEIL(12.3); -- 13
FLOOR()	Returns the largest integer less than or equal to the number.	SELECT FLOOR(12.8); -- 12
ABS() |x|	Returns the absolute value (without sign).	SELECT ABS(-25); -- 25
MOD() %	Returns the remainder after division.	SELECT MOD(10, 3); -- 1

Quick Tips

- Aggregate functions ignore NULL values (except COUNT(*)).
- Use alias with functions for better readability.
  Example: SELECT AVG(marks) AS average_marks FROM Students;
- Functions can be nested. Example: SELECT UPPER(TRIM(name)) FROM Users;

Functions = Power + Simplicity = Efficient Queries!

Date, Time & NULL Functions

Used to work with date, time and handle missing values (NULL) in SQL.
Very useful in real-world applications.

1. DATE Functions

- Work with DATE values (YYYY-MM-DD)
- CURDATE() → Returns current date
- YEAR(date) → Extract year
- MONTH(date) → Extract month (1-12)
- DAY(date) → Extract day (1-31)
- DATE_FORMAT(date, fmt) → Format date as per format



Example

```
SELECT CURDATE();                          → 2025-08-16
SELECT YEAR('2025-08-16');                 → 2025
SELECT MONTH('2025-08-16');                → 8
SELECT DAY('2025-08-16');                  → 16
SELECT DATE_FORMAT('2025-08-16', '%d-%m-%Y');
                                           → 16-08-2025
```

2. TIME Functions

- Work with TIME values (HH:MM:SS)
- CURTIME() → Returns current time
- HOUR(time) → Extract hour
- MINUTE(time) → Extract minute
- SECOND(time) → Extract second



Example

```
SELECT CURTIME();                  → 14:35:20
SELECT HOUR('14:35:20');           → 14
SELECT MINUTE('14:35:20');         → 35
SELECT SECOND('14:35:20');         → 20
```

3. DATETIME Functions

- Work with DATETIME values (YYYY-MM-DD HH:MM:SS)
- NOW() → Current date & time
- YEAR(datetime) → Year part
- MONTH(datetime) → Month part
- DAY(datetime) → Date part
- DATE_FORMAT(dt, fmt) → Format datetime



Example

```
SELECT NOW();                      → 2025-08-16 14:35:20
SELECT YEAR(NOW());                → 2025
SELECT MONTH(NOW());               → 8
SELECT DAY(NOW());                 → 16
SELECT DATE_FORMAT(NOW(),'%d-%m-%Y %H:%i');
                                   → 16-08-2025 14:35
```

4. DATEDIFF()

- Returns the difference between two dates.

DATEDIFF(date1, date2) → date1 - date2 (in days)



Example

```
SELECT DATEDIFF('2025-08-16', '2025-08-01'); → 15
('2025-08-16' minus '2025-08-01' = 15 days)
```

5. TIMESTAMP Functions

TIMESTAMP is a combination of DATE and TIME.

- CURRENT_TIMESTAMP() → Current timestamp
- UNIX_TIMESTAMP() → Seconds since 1970-01-01 00:00:00 UTC
- FROM_UNIXTIME(ts) → Convert unix timestamp to datetime

Example

```
SELECT CURRENT_TIMESTAMP();
    → 2025-08-16 14:35:20
SELECT UNIX_TIMESTAMP();
    → 1755345320 (example)
SELECT FROM_UNIXTIME(1755345320);
    → 2025-08-16 14:35:20
```



6. NULL Functions

Used to handle NULL (missing) values.

A. IFNULL(value, alt_value)

- If value is NULL, return alt_value, else return value.

Example

```
SELECT IFNULL(NULL, 'N/A');   → 'N/A'
SELECT IFNULL('John', 'N/A'); → 'John'
```

B. COALESCE(value1, value2, ...)

- Returns the first non-NULL value from the list.

Example

```
SELECT COALESCE(NULL, NULL, 'Data', 'N/A');
        → 'Data'
```

C. NULLIF(value1, value2)

- If value1 = value2, returns NULL, else returns value1.

Example

```
SELECT NULLIF(10, 10);  → NULL
SELECT NULLIF(10, 5);   → 10
```

7. NULL Handling



- NULL means unknown / missing value.
- Comparisons with NULL using =, <, > return UNKNOWN.
- Use IS NULL / IS NOT NULL to check.

Check	Example
IS NULL	SELECT * FROM emp WHERE salary IS NULL;
IS NOT NULL	SELECT * FROM emp WHERE salary IS NOT NULL;

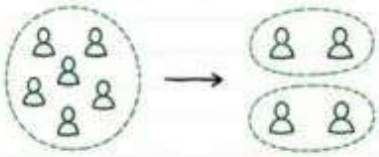
Practice is the key to Mastery ☆

GROUP BY & HAVING

- GROUP BY is used to group rows that have the same values in specified columns.
- HAVING is used to filter groups based on aggregate conditions.

1. GROUP BY

- Groups rows with same values in one or more columns.
- Used with aggregate functions like COUNT(), SUM(), AVG(), MIN(), MAX().



2. Grouping One Column

Example Table: sales

id	name	category	amount
1	Asha	Books	250
2	Bina	Books	150
3	Chetan	Pen	50
4	Dev	Books	300
5	Esha	Pen	100

Query:

```
SELECT category, COUNT(*) AS total_items
FROM sales
GROUP BY category;
```

Result:

category	total_items
Books	3
Pen	2

3. Grouping Multiple Columns

Example Table: sales

id	name	category	region	amount
1	Asha	Books	North	250
2	Bina	Books	South	150
3	Chetan	Pen	North	50
4	Dev	Books	North	300
5	Esha	Pen	South	100

Query:

```
SELECT category, region, SUM(amount) AS total_amt
FROM sales
GROUP BY category, region;
```

Result:

category	region	total_amt
Books	North	550
Books	South	150
Pen	North	50
Pen	South	100

4. Aggregate Functions with GROUP BY

Common Aggregates:

- COUNT() → counts rows
- SUM() → total of values
- AVG() → average
- MIN() → minimum
- MAX() → maximum

Example:

```
SELECT category,
       SUM(amount) AS total_amt,
       AVG(amount) AS avg_amt,
       COUNT(*) AS cnt
FROM sales
GROUP BY category;
```

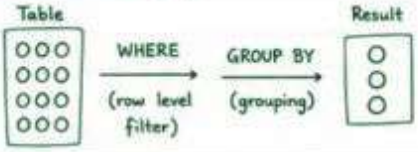
SUM, AVG, COUNT, MIN, MAX

5. WHERE with GROUP BY

- WHERE filters rows before grouping.
- Works on individual rows.

Query:

```
SELECT category, SUM(amount) AS total_amt
FROM sales
WHERE amount > 100
GROUP BY category;
```

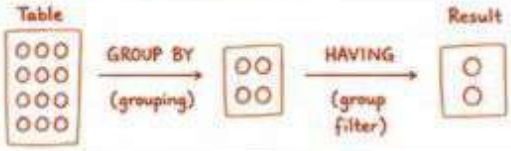


6. HAVING

- HAVING filters groups after grouping.
- Works on aggregate results.

Query:

```
SELECT category, SUM(amount) AS total_amt
FROM sales
GROUP BY category
HAVING SUM(amount) > 300;
```



7. HAVING with Aggregate Functions

- Common use: conditions on aggregates.
- Example: groups with total sales > 300

Query:

```
SELECT category, SUM(amount) AS total_amt
FROM sales
GROUP BY category
HAVING SUM(amount) > 300;
```

Result:

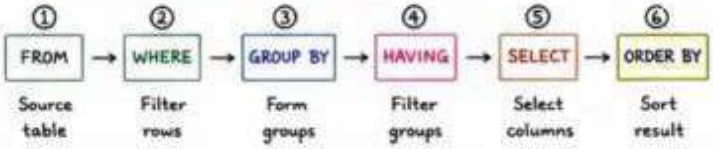
category	total_amt
Books	700

Condition applied on aggregate value

8. WHERE vs HAVING

Feature	WHERE	HAVING
Filters	Rows	Groups
When applied	Before GROUP BY	After GROUP BY
Can use aggregate functions?	No	Yes
Example	WHERE amount > 100 (row level filter)	HAVING SUM(amount) > 300 (group level filter)

9. GROUP BY Execution Concepts



☆ Remember: WHERE → rows before grouping
HAVING → groups after grouping

10. Common GROUP BY Patterns

☆ Count items per category	`SELECT category, COUNT(*) FROM table GROUP BY category;`
☆ Total sales per day	`SELECT sale_date, SUM(amount) FROM sales GROUP BY sale_date;`
☆ Average salary per department	`SELECT dept_id, AVG(salary) FROM emp GROUP BY dept_id;`
☆ Top categories by total sales	`SELECT category, SUM(amount) AS total_amt FROM sales GROUP BY category HAVING SUM(amount) > 500;`
☆ Multiple column summary	`SELECT dept_id, job_id, COUNT(*) FROM emp GROUP BY dept_id, job_id;`

CONSTRAINTS & KEYS

1. WHAT ARE CONSTRAINTS?

- Constraints are rules applied to table columns to ensure data accuracy, consistency and reliability.
- They restrict the type of data that can be inserted, updated or deleted.

2. CONSTRAINTS IN SQL



PRIMARY KEY
- Uniquely identifies each row.
- Not NULL. Only one per table.



FOREIGN KEY
- Refers to the PRIMARY KEY in another table.
- Enforces referential integrity.



UNIQUE
- Ensures all values in a column (or set of columns) are unique.



NOT NULL
- Ensures a column cannot have NULL value.



DEFAULT
- Sets a default value for a column if no value is provided.



CHECK
- Ensures values satisfy a specific condition.

3. KEYS IN DBMS



CANDIDATE KEY
- A minimal set of column(s) that can uniquely identify rows.



SUPER KEY
- Any set of column(s) that can uniquely identify rows. (May contain extra columns)



ALTERNATE KEY
- Candidate keys other than the primary key.



COMPOSITE KEY
- A key made up of two or more columns.



SURROGATE KEY
- A system-generated key with no business meaning (e.g., ID).



Keys are for IDENTIFICATION.
Constraints are for VALIDATION.

4. KEYS - EXAMPLES

StudentID	Email	Phone	Name	DeptID
101	a@x.com	9991112222	Asha	10
102	b@x.com	9993334444	Ravi	20
103	c@x.com	9995556666	Neha	10

PRIMARY KEY | UNIQUE | UNIQUE | FOREIGN KEY (DeptID in Department table)

5. REFERENTIAL INTEGRITY

- Ensures that a FOREIGN KEY value in a child table must match a PRIMARY KEY value in the parent table.
- Prevents invalid or orphan records.

DEPARTMENT (Parent Table)

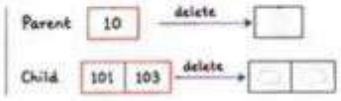
DeptID	DeptName
10	HR
20	IT
30	Finance

STUDENT (Child Table)

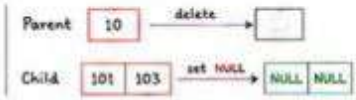
StudentID	Name	DeptID
101	Asha	10
102	Ravi	20
103	Neha	10
104	Kiran	30

6. ON DELETE (For FOREIGN KEY)

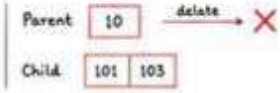
- CASCADE → When parent row is deleted, matching child rows are also deleted.



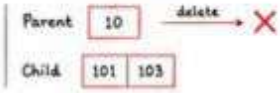
- SET NULL → When parent row is deleted, child foreign key is set to NULL.



- RESTRICT → Prevents deletion of parent row if child rows exist.

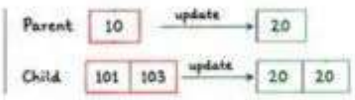


- NO ACTION → Same as RESTRICT in most databases.

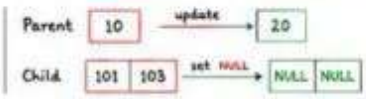


7. ON UPDATE (For FOREIGN KEY)

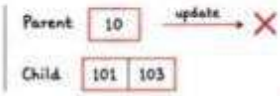
- CASCADE → When parent key is updated, child foreign key is updated automatically.



- SET NULL → When parent key is updated, child foreign key is set to NULL.



- RESTRICT / NO ACTION → Prevents update of parent key if child rows exist.



Keep Practicing! ♥

ADVANCED QUERYING

1. CASE EXPRESSION

- Used for conditional logic in SELECT, ORDER BY, WHERE, etc.
- Returns a value based on conditions.

```
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
       ...
  ELSE result
END AS alias
```

Example:

marks

id	name	score
1	Asha	85
2	Ravi	68
3	Neha	40

```
SELECT name, score,
  CASE
    WHEN score >= 75 THEN 'Distinction'
    WHEN score >= 50 THEN 'Pass'
    ELSE 'Fail'
  END AS result
FROM marks;
```

Output:

name	score	result
Asha	85	Distinction
Ravi	68	Pass
Neha	40	Fail

2. CONDITIONAL LOGIC

- Use CASE, IF(), IIF(), or boolean expressions.
- Common in projections and filtering.

Method	Example
CASE	CASE WHEN age >= 18 THEN 'Adult' ELSE 'Minor' END
IF() (MySQL)	IF(salary > 50000, 'High', 'Low')
IIF() (SQL Server)	IIF(age >= 18, 'Adult', 'Minor')
Boolean	SELECT name FROM emp WHERE active = 1;

3. SET OPERATORS

UNION

- Combines result of two queries.
- Removes duplicates.

Table A

id
1
2
3

Table B

id
3
4
5

UNION →

Result

id
1
2
3
4
5

UNION ALL

- Combines all rows.
- Keeps duplicates.

Table A

id
1
2
3

Table B

id
3
4
5

UNION ALL →

Result

id
1
2
3
4
5

INTERSECT

- Returns only common rows.

Table A

id
1
2
3

Table B

id
2
3
4

INTERSECT →

Result

id
2
3

EXCEPT (MINUS)

- Returns rows from first query that are NOT in second.

Table A

id
1
2
3

Table B

id
2
3
4

EXCEPT →

Result

id
1

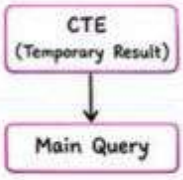
Rules:
- Both queries must have same number of columns.
- Corresponding columns must have compatible data types.
- Column names in result come from first query.

4. COMMON TABLE EXPRESSIONS (CTE)

- A temporary named result set.
- Improves readability and supports recursion.

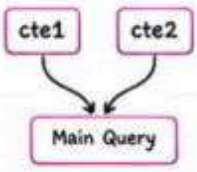
a) WITH CLAUSE (Basic CTE)

```
WITH cte_name AS (
   SELECT columns
   FROM table
   WHERE condition
)
SELECT * FROM cte_name;
```



b) MULTIPLE CTEs

```
WITH cte1 AS (SELECT * FROM emp),
   cte2 AS (SELECT dept_id, COUNT(*) cnt
            FROM emp GROUP BY dept_id)
SELECT e.name, c.cnt
FROM cte1 e
JOIN cte2 c ON e.dept_id = c.dept_id;
```



c) RECURSIVE CTEs

- Used for hierarchical / recursive data like trees.

```
WITH RECURSIVE nums(n) AS (
   SELECT 1
   UNION ALL
   SELECT n + 1 FROM nums WHERE n < 5
)
SELECT * FROM nums;
```

Output

n
1
2
3
4
5

5. CTE vs SUBQUERY

CTE	Subquery
Defined once and reused.	Written inside main query.
Improves readability.	Can make query complex.
Can be recursive.	Generally not recursive.
Used in multiple places in query.	Used in single expression.

6. CTE vs TEMPORARY TABLE

CTE	Temporary Table
Exists only for duration of query.	Exists for session (until dropped).
No storage on disk (logical result).	Stored physically in tempdb / disk.
Automatically cleaned up.	Must DROP manually.
Good for readability and short tasks.	Good for large data reuse.

QUICK RECAP

- ✓ CASE helps in conditional output.
- ✓ Set operators combine query results in different ways.
- ✓ CTEs make complex queries easier to read and maintain.
- ✓ WITH RECURSIVE handles hierarchical data.
- ✓ CTEs are temporary & query scoped; Temp tables persist for session.

Practice More, Query Better!

Database Relationships & Design

1. Entity

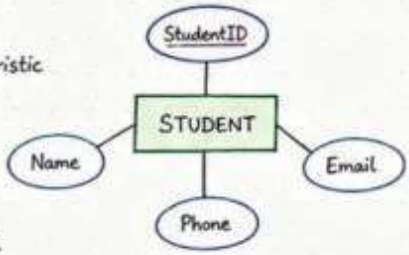
- A real-world object or thing about which data is stored.
- Represented by a rectangle in ER diagram.



Example:
Student, Course, Order

2. Attribute

- Property or characteristic of an entity.
- Represented by an oval in ER diagram.



Example:
StudentID, Name, Email

3. Relationship

- Association between two or more entities.
- Represented by a diamond in ER diagram.



Example:
Student ENROLLS in Course

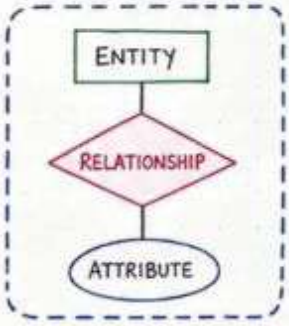
4. Cardinality

- Defines the number of instances of one entity that can be associated with instances of another entity.

Common Symbols	
1	One
N	Many
0..1	Zero or One
1..N	One or Many

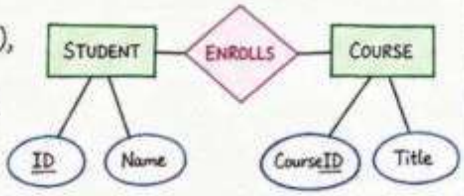
5. ER Model

- A high-level conceptual model used to describe data, entities, attributes and relationships.
- Helps in designing databases.



6. ER Diagram

- Graphical representation of ER model.
- Uses rectangles (entities), ovals (attributes) and diamonds (relationships).



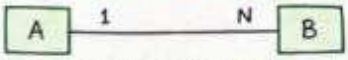
7. Types of Relationships

(a) One-To-One (1:1)



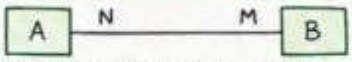
- One record in A is related to one record in B and vice-versa.
- Example: Person ↔ Passport

(b) One-To-Many (1:N)



- One record in A is related to many records in B.
- Example: Department → Employees

(c) Many-To-Many (N:M)

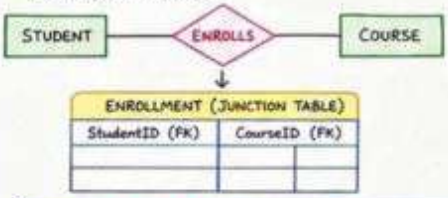


- Many records in A are related to many records in B.
- Example: Students ↔ Courses

8. Junction Tables

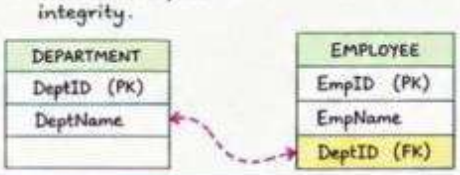
- Used to resolve Many-to-Many relationships.
- Contains foreign keys of both entities.

Example:
Students ↔ Courses



9. Foreign Key Relationships

- A foreign key is a field in one table that refers to the primary key of another table.
- Maintains referential integrity.



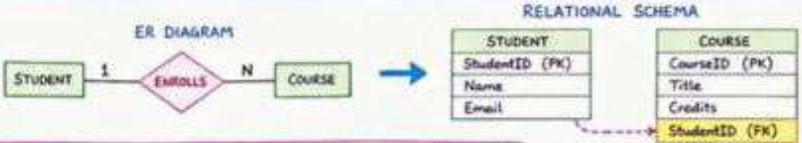
10. Schema Design

- Process of designing the overall structure of the database.
- Steps:
  1. Identify entities
  2. Identify attributes
  3. Identify relationships
  4. Define cardinality
  5. Map to relational schema



11. Relational Modeling

- Converts ER model into relational schema (tables).
- Each entity becomes a table.
- Attributes become columns.
- Relationships implemented using foreign keys.



Good database design = Less redundancy + Better integrity + Easy to maintain + Scalable

JOINS

1. What is a JOIN?

- JOIN is used to combine rows from two or more tables based on a related column.
- Related column is called the JOIN condition.

Table A (Emp)

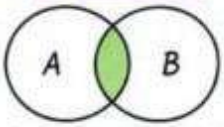
ID	Name	DeptID
1	Asha	10
2	Ravi	20
3	Meena	10

Related Column

Table B (Dept)

DeptID	DeptName
10	HR
20	IT
30	Finance

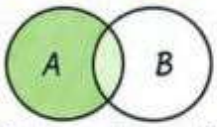
2. INNER JOIN



- Returns only matching rows from both tables.

```
SELECT *
FROM A
INNER JOIN B
ON A.id = B.id;
```

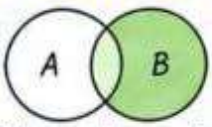
3. LEFT JOIN



- Returns all rows from left table (A) and matching rows from right table (B).

```
SELECT *
FROM A
LEFT JOIN B
ON A.id = B.id;
```

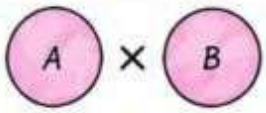
4. RIGHT JOIN



- Returns all rows from right table (B) and matching rows from left table (A).

```
SELECT *
FROM A
RIGHT JOIN B
ON A.id = B.id;
```

5. CROSS JOIN



- Returns Cartesian product (all combinations).
- No condition is used.

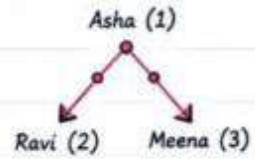
```
SELECT *
FROM A
CROSS JOIN B;
```

6. SELF JOIN

- A table joined with itself.

Employee

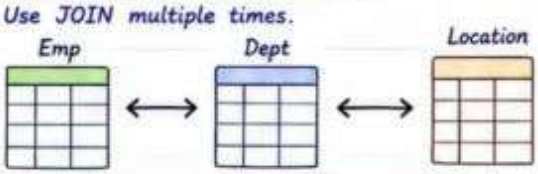
ID	Name	ManagerID
1	Asha	NULL
2	Ravi	1
3	Meena	1




```
SELECT e1.Name AS Employee, e2.Name AS Manager
FROM Employee e1
LEFT JOIN Employee e2
ON e1.ManagerID = e2.ID;
```

7. Joining Multiple Tables

- Use JOIN multiple times.




```
SELECT e.Name, d.DeptName, l.Location
FROM Emp e
JOIN Dept d ON e.DeptID = d.DeptID
JOIN Location l ON d.LocationID = l.LocationID;
```

8. JOIN Conditions

- Use ON clause to specify the joining condition.
- Common operators: =, <>, >, <, >=, <=
- Can also use multiple conditions.

```
SELECT *
FROM A
JOIN B ON A.id = B.id
AND A.status = 'Active';
```

9. JOIN with WHERE

- WHERE filters the result AFTER the join is performed.
- Used to apply conditions on joined data.

```
SELECT e.Name, d.DeptName
FROM Emp e
JOIN Dept d ON e.DeptID = d.DeptID
WHERE d.DeptName = 'IT';
```

ON → for joining
WHERE → for filtering

10. JOIN with GROUP BY

- Group the joined result based on a column.
- Used with aggregate functions.

```
SELECT d.DeptName, COUNT(*) AS EmpCount
FROM Emp e
JOIN Dept d ON e.DeptID = d.DeptID
GROUP BY d.DeptName;
```

11. JOIN with Aggregate Functions

- Aggregate functions work on the joined data.
- Common: COUNT(), SUM(), AVG(), MAX(), MIN()

```
SELECT d.DeptName, AVG(e.Salary) AS AvgSalary,
COUNT(*) AS EmpCount
FROM Emp e
JOIN Dept d ON e.DeptID = d.DeptID
GROUP BY d.DeptName;
```

12. JOIN vs Subquery

JOIN	SUBQUERY
Combines data from multiple tables in a single step.	Uses a query inside another query.
Generally faster and more efficient.	Can be slower, especially with large data.
Better for retrieving related data.	Useful for filtering, calculation or temporary result sets.
More readable for complex relationships.	Better for simple or nested conditions.

Key Takeaway

Use JOINS to combine related data efficiently. Choose the right type of JOIN based on the requirement and always use proper conditions for accurate results.

SUBQUERIES

A subquery is a query nested inside another query.

1. WHAT IS A SUBQUERY?

- A subquery (inner query) is a query inside another query (outer query).
- The inner query executes first and its result is used by the outer query.

Outer Query

```
SELECT ...
FROM ...
WHERE ... (subquery)
```

→ **Subquery (Inner Query)**

```
SELECT ...
FROM ...
WHERE ...
```

2. TYPES OF SUBQUERIES

SCALAR SUBQUERY

Returns a single value (single row, single column).

Used with =, <, >, <=, >=, <>

Example:

Find employees who earn more than the average salary.

```
SELECT * FROM emp
WHERE salary >
  (SELECT AVG(salary)
   FROM emp);
```

Result

52000

SINGLE-ROW SUBQUERY

Returns one row (can have multiple columns).

Used with =, <, >, <=, >=, <>

Example:

Find employee whose salary is same as John's.

```
SELECT * FROM emp
WHERE (id, salary) =
  (SELECT id, salary
   FROM emp
   WHERE name = 'John');
```

id	name	salary
3	John	50000

MULTI-ROW SUBQUERY

Returns multiple rows (only one column).

Used with IN, ANY, ALL, EXISTS.

Example:

Find employees working in departments located in 'Delhi' or 'Mumbai'.

```
SELECT * FROM emp
WHERE dept_id IN
  (SELECT id FROM dept
   WHERE location IN
   ('Delhi', 'Mumbai'));
```

id
1
2
5
...

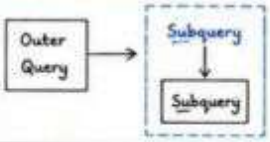
NESTED SUBQUERY

A subquery inside another subquery.

Example:

Find employees who earn more than the average salary of their department.

```
SELECT * FROM emp e
WHERE salary >
  (SELECT AVG(salary)
   FROM emp
   WHERE dept_id = e.dept_id);
```



CORRELATED SUBQUERY

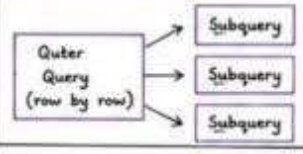
Subquery depends on a column from the outer query.

Executes once for each row of outer query.

Example:

Find employees who earn more than the average salary of their department.

```
SELECT * FROM emp e
WHERE salary >
  (SELECT AVG(salary)
   FROM emp
   WHERE dept_id = e.dept_id);
```



3. SUBQUERY IN DIFFERENT CLAUSES

IN SELECT

Subquery used to return a value in the SELECT list.

Example:

```
SELECT name,
  (SELECT COUNT(*)
   FROM emp e2
   WHERE e2.dept_id = e.dept_id)
  AS dept_emp_count
FROM emp e;
```

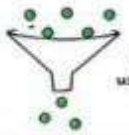
name	dept_emp_count
John	3
Alice	2
...	

IN WHERE

Subquery used to filter rows in WHERE clause.

Example:

```
SELECT * FROM emp
WHERE salary >
  (SELECT AVG(salary)
   FROM emp);
```



Filter rows using subquery result

IN FROM

Subquery used as a table in FROM clause.

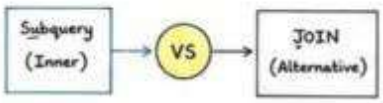
Example:

```
SELECT d.dept_name, d.total_emp
FROM (SELECT dept_id, COUNT(*)
        AS total_emp
      FROM emp
      GROUP BY dept_id) d;
```

dept_name	total_emp
HR	3
IT	5
Sales	2
...	

SUBQUERY WITH JOIN (Alternative)

Sometimes JOIN can replace subquery for better performance.



JOIN is usually faster for large datasets.

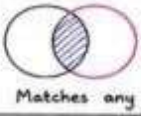
4. SPECIAL OPERATORS WITH SUBQUERIES

IN

Matches any value in the list returned by subquery.

Example:

```
WHERE dept_id IN
  (SELECT id FROM dept);
```



Matches any

EXISTS

Returns true if subquery returns at least one row.

Example:

```
WHERE EXISTS
  (SELECT 1 FROM emp e2
   WHERE e2.dept_id = e.dept_id);
```



At least one exists

NOT EXISTS

Returns true if subquery returns no rows.

Example:

```
WHERE NOT EXISTS
  (SELECT 1 FROM emp e2
   WHERE e2.dept_id = e.dept_id);
```



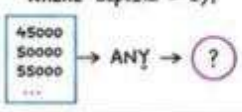
No rows exist

ANY

Compare a value with any value returned by subquery.

Example:

```
WHERE salary > ANY
  (SELECT salary FROM emp
   WHERE dept_id = 2);
```

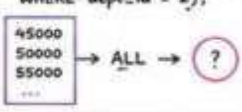


ALL

Compare a value with all values returned by subquery.

Example:

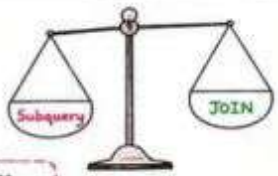
```
WHERE salary > ALL
  (SELECT salary FROM emp
   WHERE dept_id = 2);
```



5. SUBQUERY vs JOIN

SUBQUERY

- Easy to write for complex conditions.
- Good for readability in some cases.
- May be slower on large datasets (depends on DB engine).
- Best for single value or small result sets.



JOIN

- Generally faster for large datasets.
- Better performance with indexing.
- Preferred for combining data from multiple tables.
- Good for one-to-many and many-to-many relationships.

TIP: Use subqueries when the logic is easier to express. Use JOINs when performance is critical on large data.

Happy Querying! ☺

★ Normalization ★

1. What is Normalization?

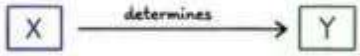
- Normalization is a process of organizing data in a database to reduce redundancy and improve data integrity.
- It involves dividing large tables into smaller tables and defining relationships between them.



Goal: Store each fact only once, minimize redundancy and avoid anomalies (insert, update, delete).

2. Functional Dependency (FD)

- In a relation R, attribute Y is functionally dependent on X (written as X → Y) if each value of X has exactly one value of Y.



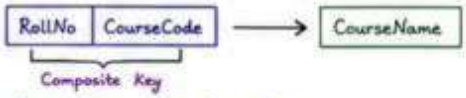
Example:

RollNo → StudentName
RollNo → Dept

} RollNo determines StudentName and Dept.

3. Partial Dependency

- When an attribute depends on part of a composite key.



CourseName depends only on CourseCode (part of the key).

4. Transitive Dependency

- When an attribute depends on another non-key attribute.



Here, RollNo → DeptId and DeptId → DeptName
So, RollNo → DeptName (transitive)

5. 1NF (First Normal Form)

Criteria:

- All attributes are atomic (indivisible).
- No repeating groups / arrays.

Example:

✗ Not in 1NF

StudentID	Name	Phones
1	Asha	{9876, 9123}

✓ In 1NF

StudentID	Name	Phone
1	Asha	9876
1	Asha	9123

6. 2NF (Second Normal Form)

Criteria:

- In 1NF.
- No partial dependency. Every non-key attribute must be fully dependent on the whole primary key.

Example:

✗ Not in 2NF

RollNo	CourseCode	CourseName
1	CS101	DBMS

CourseName depends only on CourseCode (partial dependency).

✓ In 2NF (decomposed)

Enrollments

RollNo	CourseCode
1	CS101

Courses

CourseCode	CourseName
CS101	DBMS

7. 3NF (Third Normal Form)

Criteria:

- In 2NF.
- No transitive dependency. Non-key attributes should not depend on other non-key attributes.

Example:

✗ Not in 3NF

RollNo	DeptId	DeptName
1	D1	IT

DeptName depends on DeptId (not on RollNo directly).

✓ In 3NF (decomposed)

Student

RollNo	DeptId
1	D1

↓

Department

DeptId	DeptName
D1	IT

8. BCNF (Boyce-Codd Normal Form)

Criteria:

- In 3NF.
- For every FD (X → Y), X should be a super key.

Example:

✗ Not in BCNF

A	B	C
1	2	3
1	2	4

FD: (A,B) → C and C → B
Here C is not a super key.

✓ In BCNF (decomposed)

R1

C	B
3	2
4	2

R2

A	C
1	3
1	4

9. Normalization Examples (Summary)



10. Advantages of Normalization

- ★ Reduces data redundancy.
- ★ Improves data integrity and consistency.
- ★ Avoids insert, update and delete anomalies.
- ★ Saves storage space.
- ★ Easier maintenance.
- ★ Clear relationships between tables.



11. Denormalization

- The process of combining normalized tables to improve read performance.
- Introduces some redundancy intentionally.
- Useful in reporting, data warehouses and when read >> write operations.



12. Normalization vs Denormalization

Aspect	Normalization	Denormalization
Goal	Reduce redundancy and improve integrity	Improve performance, reduce joins
Redundancy	Less redundancy	More redundancy.
Performance (Read)	Slower (more joins)	Faster
Performance (Write)	Faster	Slower (due to redundancy)
Storage	Less storage	More storage
Use Case	OLTP, operational systems	Reporting, analytics, data warehouses

Remember!

Normalize up to 3NF (or BCNF if required).
Denormalize only when performance demands it.

☺

Views & Temporary Tables

1. What is a View?

- A view is a virtual table based on the result of a SELECT query.
- It does not store data physically.
- It shows data from one or more tables.

Table: Employees

id	name	dept_id
1	Asha	10
2	Ravi	20
3	Neha	10

Table: Dept

dept_id	dept_name
10	HR
20	IT



Result

name	dept_name
Asha	HR
Ravi	IT
Neha	HR

2. Create View

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Example:

```
CREATE VIEW emp_view AS
SELECT e.id, e.name, d.dept_name
FROM employees e
JOIN dept d ON e.dept_id = d.dept_id;
```

3. Alter View

```
ALTER VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Example:

```
ALTER VIEW emp_view AS
SELECT e.id, e.name, e.salary, d.dept_name
FROM employees e
JOIN dept d ON e.dept_id = d.dept_id;
```

4. Drop View

```
DROP VIEW view_name;
```

Example:

```
DROP VIEW emp_view;
```

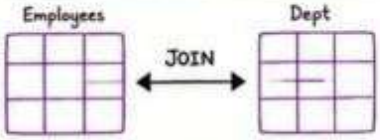
View is removed.



5. Views with Joins

- Used to combine data from multiple tables.

```
CREATE VIEW emp_dept_view AS
SELECT e.name, d.dept_name, e.salary
FROM employees e
JOIN dept d ON e.dept_id = d.dept_id;
```



6. Views with Aggregations

- Used for summary reports.

```
CREATE VIEW dept_summary AS
SELECT dept_id, COUNT(*) AS total_emp,
       AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id;
```



7. Updatable Views

- Some views allow INSERT, UPDATE, DELETE operations.

Conditions:

1. Based on a single table.
2. No DISTINCT, GROUP BY, aggregates, or JOINs.
3. Includes primary key.

8. Advantages of Views

- ✓ Simplify complex queries.
- ✓ Provide security (hide sensitive data).
- ✓ Data independence (changes in base tables don't affect applications using the view).
- ✓ Reusability of queries.

9. Limitations of Views

- ✗ May be slower than direct table access.
- ✗ Cannot always be updated.
- ✗ Complex views can be difficult to understand.
- ✗ Depend on base tables (if base table dropped, view becomes invalid).

Temporary Tables

10. Temporary Tables

- Temporary tables exist only for the duration of the session/connection.
- They are stored in tempdb (SQL Server) or in-memory.
- Automatically dropped when the session ends.

11. Create Temporary Table

```
CREATE TEMPORARY TABLE temp_emp (
  emp_id INT,
  emp_name VARCHAR(50),
  salary DECIMAL(10,2)
);
```

Example:

```
INSERT INTO temp_emp (emp_id, emp_name, salary)
SELECT emp_id, emp_name, salary
FROM employees
WHERE salary > 50000;
```

12. Temporary vs Permanent Tables

Feature	Temporary Table	Permanent Table
Existence	Only for session	Until dropped
Storage	tempdb / In-memory	Database storage
Name	Starts with # (local) or ##) or ##	User-defined name
Visibility	Visible only in current session	Visible to all sessions
Auto Drop	Yes	No

Note: Use VIEWS for abstraction and security.
Use TEMPORARY TABLES for intermediate results and performance.

Indexes & Performance

What is an Index?

- An index is a special data structure that improves the speed of data retrieval operations on a table.
- Think of it like the index of a book – it helps find data quickly without scanning the whole table.



How Indexes Work

Indexes store a sorted list of key values with pointers (row locations) to the actual data.

Index (on id)

id	Pointer
101	→ Row 1
102	→ Row 2
103	→ Row 3
104	→ Row 4
...	

Table: students

id	name	age
101	Aisha	20
102	Ravi	21
103	Meena	19
104	Arjun	22
...	...	

Primary Index

- Built on the PRIMARY KEY column.
- Automatically created.



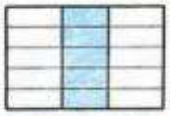
Unique Index

- Ensures all values in the indexed column(s) are unique.
- Duplicate values not allowed.



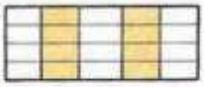
Single-Column Index

- Index on a single column.
- Improves search on that column.



Composite (Multi-Column) Index

- Index on multiple columns.
- Useful for queries filtering/sorting on those columns.



Full-Text Index

- Used to search text data (words or phrases).
- Fast text search in large text fields.



Creating Indexes

```
-- Syntax
CREATE INDEX index_name
  ON table_name (column_name);
```

Examples:

- Single-Column Index
  `CREATE INDEX idx_name ON students(name);`
- Composite Index
  `CREATE INDEX idx_name_age ON students(name, age);`
- Unique Index
  `CREATE UNIQUE INDEX idx_email ON users(email);`

Removing Indexes

```
-- Syntax
DROP INDEX index_name ON table_name;
```

Example:

```
DROP INDEX idx_name ON students;
```



Index Advantages

- ✓ Faster data retrieval (SELECT queries).
- ✓ Improves performance of JOINs.
- ✓ Helpful in ORDER BY, GROUP BY, and WHERE clauses.
- ✓ Enforces uniqueness (with UNIQUE index).



Index Disadvantages

- ✗ Slower write operations (INSERT, UPDATE, DELETE).
- ✗ Consumes extra storage space.
- ✗ Too many indexes can degrade performance.
- ✗ Overhead in maintaining indexes.



When to Use Indexes

- Columns used frequently in WHERE, JOIN, ORDER BY, GROUP BY.
- Columns with high selectivity (many unique values).
- Large tables with many rows.
- Columns used for searching and sorting.



When Not to Use Indexes

- Small tables (overhead > benefit).
- Columns with low selectivity (few unique values).
- Columns updated very frequently.
- When disk space is very limited.



Composite Index Order

- In a composite index (A, B, C), the order of columns matters.
- MySQL can use the index for queries starting from the leftmost column.
- Best Practice: Place the most selective column first.
- Example:
  `CREATE INDEX idx_ab ON orders(customer_id, order_date);`

Index: (customer_id, order_date)

customer_id	order_date	Pointer
101	2024-01-01	→ Row 1
101	2024-01-05	→ Row 2
102	2024-01-02	→ Row 3
...		

Usable For Queries:

- ✓ WHERE customer_id = 101
- ✓ WHERE customer_id = 101 AND order_date = '2024-01-05'
- ✓ ORDER BY customer_id, order_date

Not Ideal:

- ✗ WHERE order_date = '2024-01-01'

Stored Programs & Triggers

Stored programs are precompiled SQL code stored in the DB.
They improve reusability, security and performance.



1) Stored Procedures

- A stored procedure is a set of SQL statements stored in the database.
- It can accept parameters and return values.
- **Create Procedure**

```
CREATE PROCEDURE proc_name (param_list)
AS
BEGIN
    -- SQL statements --
END;
```

- **Procedure Parameters**

Type	Meaning
IN	Pass value into procedure (default mode) →
OUT	Return value from procedure to caller ←
INOUT	Pass value in and return modified value ↔

2) Stored Functions

- A function returns a single value.
- It can be used in SQL expressions.
- **Return**

```
CREATE FUNCTION func_name (param_list)
RETURNS data_type
AS
BEGIN
   -- SQL statements --
  RETURN value;
END;
```

3) Procedure vs Function

Feature	Procedure	Function
Returns	0 or more values	Exactly one value
Used in	CALL statement	In expressions/queries
Parameters	IN / OUT / INOUT	IN only
Purpose	Perform actions	Return calculated value

4) Triggers

- A trigger is a stored program that automatically executes when an event occurs on a table.
- It helps in enforcing rules, maintaining audit logs, etc.

Trigger Type	Event	When it Fires
BEFORE INSERT	INSERT	Before a new row is inserted
AFTER INSERT	INSERT	After a new row is inserted
BEFORE UPDATE	UPDATE	Before existing row is updated
AFTER UPDATE	UPDATE	After existing row is updated
BEFORE DELETE	DELETE	Before a row is deleted
AFTER DELETE	DELETE	After a row is deleted

Example : AFTER INSERT Trigger

```
CREATE TRIGGER trg_after_insert
AFTER INSERT ON emp
FOR EACH ROW
BEGIN
  INSERT INTO emp_audit(emp_id, action, log_time)
   VALUES (NEW.emp_id, 'INSERT', NOW());
END;
```

5) Trigger Limitations



- Can make debugging harder.
- May affect performance if overused.
- Order of trigger execution is not guaranteed if multiple triggers exist for same event.
- Cannot be called directly (fires automatically).

Note

Stored programs make code modular, secure and reusable.
Triggers enforce rules automatically.
Cursors help in row-by-row processing.

6) Cursors

- A cursor is used to process rows one at a time.
- Useful when we need row-by-row processing.
- **Cursor Lifecycle**



1. **DECLARE** — Define cursor with a query
2. **OPEN** — Allocate resources and execute query
3. **FETCH** — Retrieve one row at a time into variables
4. **EXIT** — When no more rows are available
5. **CLOSE** — Release cursor resources

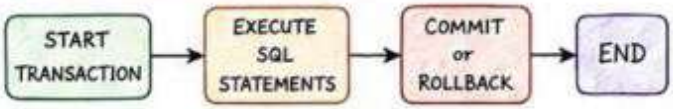
★ Transactions & Concurrency ★

1. What is a Transaction?

- A transaction is a sequence of one or more SQL operations treated as a single unit of work.
- Either all the operations succeed (COMMIT) or none of them (ROLLBACK).



2. Transaction Lifecycle



3. START TRANSACTION

- Marks the beginning of a transaction.

```
START TRANSACTION;
```

4. COMMIT

- Saves all changes permanently.

```
COMMIT;
```

5. ROLLBACK

- Undoes all changes made in the transaction.

```
ROLLBACK;
```

6. SAVEPOINT

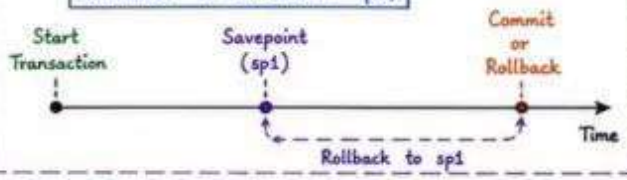
- Sets a point within a transaction to which we can later roll back.

```
SAVEPOINT sp1;
```

7. ROLLBACK TO SAVEPOINT

- Rolls back the transaction to a specific savepoint.

```
ROLLBACK TO SAVEPOINT sp1;
```



8. ACID Properties

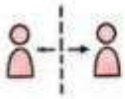
A — Atomicity
- All operations in a transaction are executed completely or not at all.



C — Consistency
- Ensures that a transaction brings the database from one valid state to another.



I — Isolation
- Concurrent transactions do not interfere with each other. Intermediate results are hidden from others.

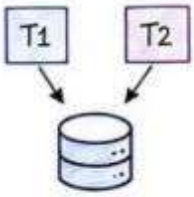


D — Durability
- Once committed, changes are permanent even in case of system failure.



9. Concurrency

- When multiple transactions run at the same time on the same data.
- Concurrency control ensures data correctness and consistency.



10. Locking

- Locks are used to control access to data.

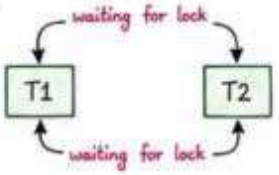
Lock Type	Purpose
Shared Lock (S)	Allows read access. Multiple transactions can hold it.
Exclusive Lock (X)	Allows read and write. Only one transaction can hold it.





11. Deadlocks

- Occurs when two or more transactions are waiting for each other to release locks.
- DBMS detects and resolves deadlocks (e.g., aborts one transaction).



12. Isolation Levels

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Description
READ UNCOMMITTED	Possible ✗	Possible ✗	Possible ✗	Lowest isolation. Can read uncommitted data.
READ COMMITTED	Not Possible ✓	Possible ✗	Possible ✗	Prevents dirty reads. Others can modify after read.
REPEATABLE READ	Not Possible ✓	Not Possible ✓	Possible ✗	Same rows won't change, but new rows may appear.
SERIALIZABLE	Not Possible ✓	Not Possible ✓	Not Possible ✓	Highest isolation. Transactions run serially.

Remember: Use transactions to ensure reliability. Choose appropriate isolation level for performance and consistency.